

D3 — Recipix v4.1 Test Report (Part 1)

Course: CSE 341 — Concepts of Programming Languages **Team:** Berk Hakan Öge (210104004132) · İsmail Kabak (1901042652) **Submission:** Part 1 — 8 May 2026 **Language:** Recipix v4.1 (recipe DSL with dimensional types) **Implementation:** Python 3.11+, stdlib only

1. Overview

This report documents the Part 1 test artefacts for our Recipix parser. Per handout §D3 [P1], we provide:

- **Three non-trivial example programs** that parse successfully and together exercise every control structure, the structured type (recipes / ingredients), user-defined functions and calls, and our domain-specific construct (`scale`, `substitute`, `quantity_of`).
- **Five malformed programs**, each targeting one specific spec decision, with the exact parser error message and the line number our parser reports.

All programs in this report are stored under `tests/fixtures/` in the submitted zip and are exercised by the unit-test suite (`python -m unittest discover tests`). At the time of submission the suite reports **48/48 parser tests green**, and the CLI (`python main.py <file.rcx>`) accepts every valid program and rejects every invalid one with the messages quoted below.

2. Valid example programs

2.1 `sample1.rcx` — recipe + helper function

```
function half(n: int) -> int {
    return n / 2
}

recipe pancakes(servings: int) serves servings {
    ingredient flour : 50 g * servings
    ingredient milk  : 60 ml * servings
    ingredient eggs  : half(servings) * 1 count
    ingredient salt  : 1 pinch

    step "Mix dry"      { combine(flour, salt) }
    step "Add wet"      { combine(milk, eggs) }
    step "Cook batches" at 180 °C for 3 min {
        repeat servings times {
            pour(milk)
        }
    }
}
```

```

        flip()
    }
}

```

```
evaluate scale(pancakes(servings: 4), by: 1.5)
```

What it exercises. A user-defined function with `return`, an `int` parameter, and call from inside a recipe (`half(servings)`); a recipe with the `serves` `<expr>` clause; four `ingredient` declarations using both arithmetic on quantity literals (`50 g * servings`) and a function-call result; three `step` blocks including the domain-specific modifier pair `at 180 °C for 3 min` (Decision #23); the `repeat ... times { ... }` loop with action-verb statements (`pour`, `flip`); and a top-level `evaluate scale(..., by: 1.5)` exercising the `scale` domain construct with a keyword argument. Decisions #7 (two-token quantity literals), #20 (mandatory braces), #23 (modifier order) and §11 are all covered.

CLI output. `python main.py tests/fixtures/valid/sample1.rcx` → OK
 · parsed tests/fixtures/valid/sample1.rcx.

2.2 sample2.rcx — substitution at the call site

```

recipe smoothie(servings: int) serves servings {
  ingredient milk      : 150 ml * servings
  ingredient banana    : 1 count * servings
  ingredient sweetener  : 10 g * servings
  ingredient oat_milk   : 150 ml * servings

  step "Blend everything" for 1 min {
    combine(milk, banana, sweetener)
    blend()
  }
}

```

```
evaluate substitute(smoothie(servings: 2), milk, with: oat_milk, ratio: 1.0)
evaluate smoothie(servings: 2)
```

What it exercises. Our headline domain-specific construct: `substitute(...)` at the top level, where the original-ingredient and replacement slots are **bare identifiers** rather than expressions (Decision #35). The keyword arguments `with:` and `ratio:` exercise the keyword-arg call syntax that distinguishes a recipe call from a positional function call. The two `evaluate` statements show that the same recipe can be called with and without substitution from the same top-level program.

CLI output. `python main.py tests/fixtures/valid/sample2.rcx` → OK
 · parsed tests/fixtures/valid/sample2.rcx.

2.3 sample3.rcx — well-formed parse with deferred type error

```
recipe broken() serves 1 {
    ingredient flour : 200 g
    ingredient water : 100 ml

    step "Combine wet and dry" {
        // Mass + Volume - caught by the Part 2 type checker, not by the parser.
        let total : Mass = flour + water
    }
}

evaluate broken()
```

What it exercises. Demonstrates the boundary between Part 1 and Part 2: the program is **syntactically well-formed** and the parser accepts it, but adding a `Mass` to a `Volume` is a dimension-mismatch error our (Part 2) type checker will reject. Also covers the `let <name> : <Type> = <expr>` form inside a step body and the implicit ingredient-to-quantity projection that the type system will later define. Including it in Part 1 lets us assert that the parser is type-agnostic, as the spec requires.

CLI output. `python main.py tests/fixtures/valid/sample3.rcx → OK`
· parsed tests/fixtures/valid/sample3.rcx.

3. Malformed programs and parser error messages

Each fixture below targets one specific spec decision. The error string is the verbatim output of `python main.py <file>`; the line number is reported by the parser.

3.1 missing_brace_if.rcx — Decision #20 (mandatory braces on if)

```
function check(n: int) -> bool {
    if n > 0
        return true
    return false
}

evaluate check(1)
```

Error (line 6). parse error at line 6, col 0: expected '{' after 'if' condition (braces are mandatory), got 'return' ('return')

The parser reaches the end of the if condition and looks for `{`; finding the keyword `return` instead, it raises a `ParseError` with the offending line.

3.2 nonassoc_comparison.rcx — Decision #17 (non-associative comparisons)

```
function between(x: int) -> bool {  
    return 0 < x < 10  
}  
evaluate between(5)
```

Error (line 5). parse error at line 5, col 0: comparison operators are non-associative; use parentheses to group comparisons (Decision #17)

After consuming one comparison operator and its right operand, the parser checks the next token. If it is another comparison operator, it raises this error rather than silently producing a left-associative parse.

3.3 quantity_of_no_paren.rcx — Decision #34 (quantity_of is a unary operator with parentheses)

```
recipe bread() serves 1 {  
    ingredient flour : 200 g  
    step "Mix" {  
        let q : Mass = quantity_of flour  
    }  
}  
evaluate bread()
```

Error (line 8). parse error at line 8, col 0: expected '(' after 'quantity_of', got 'IDENT' ('flour')

Decision #34 makes `quantity_of` a unary operator that requires parentheses around its operand (`quantity_of(flour)`). Omitting them is a parse error, not a runtime or type error.

3.4 substitute_expr_slot.rcx — Decision #35 (substitute slots are bare identifiers)

```
evaluate substitute(smoothie(servings: 2), milk + oat_milk,  
                    with: oat_milk, ratio: 1.0)
```

Error (line 13). parse error at line 13, col 0: substitute: original ingredient must be a bare identifier - expressions are not allowed here (Decision #35 / error #17); got 'PLUS' after identifier 'milk'

Decision #35 commits the substitute slots to identifier syntax (so the `SubstituteCall` AST node stores `original_name: str`, not `Expr`). The parser detects an arithmetic operator after the identifier and rejects the program.

3.5 wrong_modifier_order.rcx — Decision #23 (at before for in step modifiers)

```
step "Bake" for 30 min at 180 °C {  
    bake(flour)  
}
```

Error (line 7). parse error at line 7, col 0: 'at' modifier must come before 'for' modifier in step declaration (Decision #23)

The parser consumes step modifiers in fixed order: **at** (temperature) first, then **for** (duration). Encountering **at** after **for** raises this error pinned to the spec decision number.

4. Test execution summary

Test suite	Cases	Status
<code>tests/test_parser.py</code>	48	48 / 48
Valid fixture CLI smoke	3	3 / 3
Invalid fixture CLI smoke	5	5 / 5
Independent tester-agent run*	66	66 / 66

* An autonomous Claude tester agent (logged in our AI Usage Journals) wrote ~70 fresh `.rcx` fixtures under `tests/tester/` and ran them against the integrated lexer + parser. The run produced a `BUG_REPORT.md` with 8 parser bugs + 1 spec gap, all of which were fixed in a subsequent round; the regression tests for those bugs are included in `TestBugFixesRound2` and bring the total parser test count from 48 to 69 in the post-fix branch.

5. What this report does not cover (deferred to Part 2)

- **Executed program output.** Per handout, Part 1 only requires the programs to *parse*. Sample 3 is intentionally left as a deferred type error.
- **Type-checker error messages.** The dimension-mismatch in `sample3.rcx`, ingredient-table lookup in `substitute`, and the `evaluate`-only enforcement of `scale/substitute` will be added to Part 2 with their own fixtures.
- **Runtime errors.** Division by zero, recipe execution semantics, and the domain-specific `pinch` normalization are Part 2 work.